Overview

Multimedia Retrieval · Chapter 0

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

About this book

This is the companion textbook for **Multimedia Retrieval**, a Master's-level course at the University of Basel, edition HS26. Course announcements, exercise submissions, and scheduling are managed through ADAM and GitHub

From Ctrl+F to AI Answers

Suppose you need to find all books about cats in a library. A simple string search (Ctrl+F for "cat") scans every page until it finds a match. Even for a modest collection this is impractical, and the result is merely a Boolean filter: it tells you which books contain the string "cat", but it cannot tell you which ones are *most relevant* to what you are looking for. For popular topics, you are left with a long list of matches, and have to manually browse through the results to find what you wanted.

Retrieval systems solve this problem in two steps. First, they organize information in advance through preprocessing and indexing: extracting features from documents, building data structures that enable fast lookup, and storing statistics that describe each document's content. Second, at query time they apply ranking models that score documents by their relevance to the query rather than simply filtering by presence or absence of a term.

Modern AI systems have made retrieval even more critical. Large language models can in theory process long documents, but doing so is expensive (costs scale with token count) and prone to hallucination when the model must rely on memorized knowledge. In practice, it is far more effective to first retrieve the relevant passages from a document collection and feed only those into the model's context. This pattern, retrieval-augmented generation, depends entirely on the quality of the retrieval step: if the wrong passages are retrieved, the generated answer will be wrong too.

The Semantic Gap

Consider searching for a picture of a cat. Nothing in the raw pixel values tells a machine that an image depicts a cat. The system sees a grid of numbers; the user asks a question about meaning. This mismatch between how humans describe what they want and how machines represent information is called the *semantic gap* (Figure 0.1).

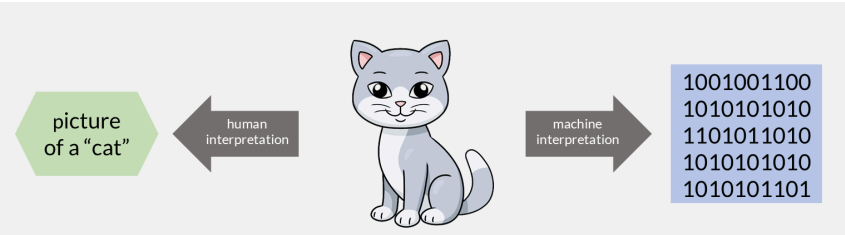

Figure 0.1: A human sees a "cat"; a machine sees a grid of binary values.

The gap exists in text too, though it is narrower. A search for "cat" will match many documents containing that word or its plural "cats", but it will miss documents that discuss "felines" or mention specific breeds by name without ever using the word "cat". In images, audio, and video the gap is far wider: there is no surface-level token to match against.

The semantic gap plagued earlier retrieval systems, especially those working with non-text media. Recent breakthroughs in AI, particularly dense embeddings and transformer models, have dramatically narrowed it by learning to map both queries and documents into a shared space where meaning, not surface form, determines similarity. This book traces the full path from systems that match keywords to systems that understand meaning.

Closing the Gap

The semantic gap is not one problem but a spectrum of problems, and this book attacks them in order of increasing sophistication.

The first step is keyword matching: building an index so that a search for “cat” instantly finds every document containing that word, without scanning the entire collection. This alone closes the gap for queries where the user’s words appear verbatim in the target. But as we saw, many relevant documents use different vocabulary.

The second step is statistical ranking. Instead of treating every keyword match equally, we weight terms by how informative they are and score documents by how well they match the query overall. A document that mentions “cat” once in passing ranks lower than one dedicated to the topic. This narrows the gap further, but only within the space of shared vocabulary.

The third step is semantic matching. We represent both queries and documents as dense vectors in a shared space, trained so that “cat” and “feline” end up near each other. Now the system can find relevant documents even when no words overlap. This is where the gap narrows dramatically for text, and where non-text media (images, audio) first become searchable by meaning.

The final step is retrieval-augmented generation. Rather than returning a ranked list of documents, the system retrieves the most relevant passages and feeds them to a language model that synthesizes a direct answer. The gap between question and answer is closed end-to-end.

Each step builds on the previous one. A RAG pipeline still needs an index (step 1), still benefits from good ranking (step 2), and works best with semantic retrieval (step 3). This is why the book is structured as a cumulative progression rather than a menu of independent topics.

This progression was not designed in one stroke. It accumulated over six decades, as each generation of researchers added a new way to narrow the gap. The optional history below traces that path from the first computerized catalogues to today’s generative systems.

From card catalogues to transformers: six decades of retrieval (optional reading)

The 1960s saw the birth of computerized information retrieval. Calvin Mooers had coined the term “information retrieval” in 1950, but serious research began only in the 1960s. H. P. Luhn created KWIC (Key Word In Context) indexes, IBM developed the STAIRS system, and Gerard Salton began work on SMART at Cornell. Most systems remained experimental because few texts existed in machine-readable form, and retrieval was still largely a human-mediated activity (Figure 0.2).

Figure 0.2: Library retrieval in the 1960s: a manual reference desk (left) beside an early terminal-based catalogue (right).

The 1970s brought the first practical systems. Widespread computer typesetting produced the machine-readable text needed for large-scale retrieval. Salton’s vector space model represented documents and queries as vectors whose similarity could be measured mathematically. Robertson and Sparck Jones developed the probabilistic framework that would later lead to BM25. Commercial systems like Lockheed Dialog and MEDLARS demonstrated that automated retrieval could work at scale (Figure 0.3).

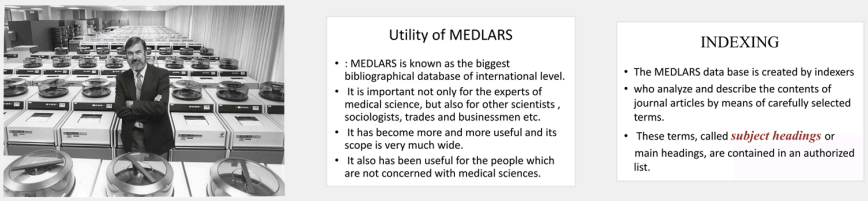

Figure 0.3: The MEDLARS system: an early large-scale bibliographic database running on reel-to-reel tape storage.

The 1980s refined these models. Boolean retrieval became the dominant commercial approach, letting users combine terms with AND, OR, and NOT (Figure 0.4). Robertson formalized the Probability Ranking Principle. The Okapi system at City University London gave its name to the BM25 ranking function. Latent Semantic Indexing appeared as a method to reveal hidden relationships between terms.

How to Search Using Boolean Operators:

Concept	Search Examples	Results
AND	politics **AND** media children **AND** poverty "civil war" **AND** Virginia	Results will include both terms
OR	"law enforcement" **OR** police labor **OR** labour 60s **OR** sixties	Results will include one or both terms
NOT	"civil war" **NOT** American Caribbean **NOT** Cuba therapy **NOT** physical	Excludes results with the term following NOT

Figure 0.4: Boolean operators (AND, OR, NOT) as set operations that select matching documents.

The 1990s brought the web revolution. Early search engines (Archie, WebCrawler, AltaVista) appeared to help users navigate the new online world (Figure 0.5). The most significant development was PageRank (Page and Brin, 1996), which ranked pages by the authority of incoming links rather than keyword matching alone.

Figure 0.5: Early web interfaces circa 1998-1999, showing keyword search boxes and directory-style navigation.

The 2000s introduced machine learning into ranking. Learning to Rank (LTR) trained supervised models to order results by relevance. BM25 became a standard baseline. Deep learning re-emerged after a quiet period, driven by more powerful GPUs and larger datasets (Figure 0.6).

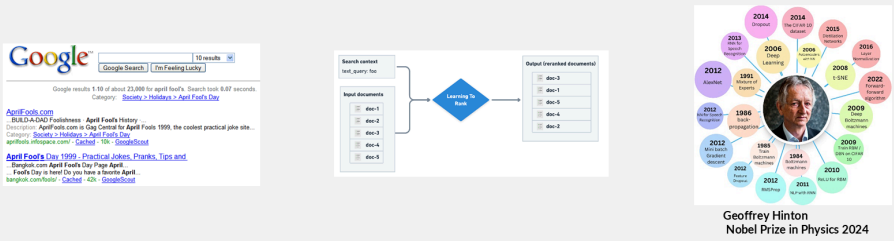

Figure 0.6: The 2000s: Google-era keyword search alongside the rise of Learning-to-Rank.

The 2010s saw deep learning applied directly to retrieval. Neural ranking models replaced handcrafted features with learned representations. Google introduced BERT in 2018, enabling search engines to understand context and word relationships. Transformer architectures fundamentally changed how machines handle sequential data (Figure 0.7).

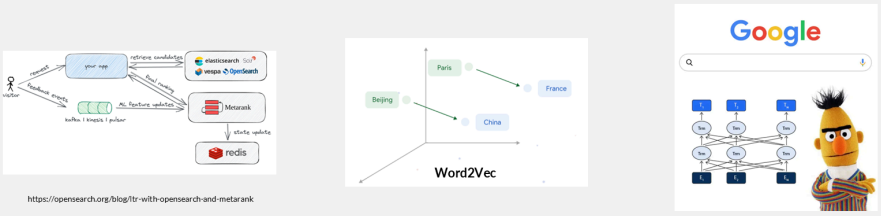

Figure 0.7: Neural ranking in the 2010s: a Learning-to-Rank pipeline (left), an embedding space (center), and contextual models like BERT (right).

The 2020s brought semantic search and vector databases as production infrastructure. Retrieval-Augmented Generation combines retrieval with large language models. Agentic systems can reason about information needs, execute multi-step retrieval, and synthesize results from multiple sources (Figure 0.8).

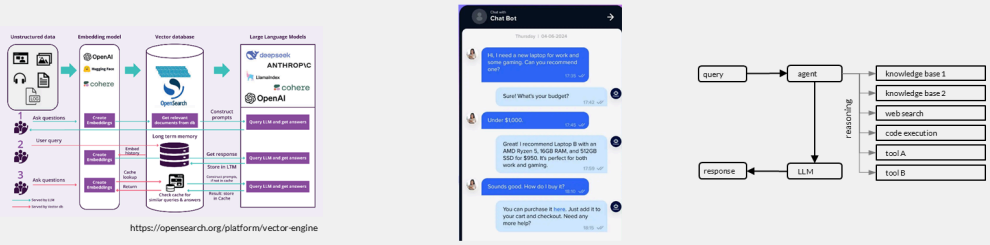

Figure 0.8: Three architectures for knowledge-grounded AI: a RAG pipeline with vector retrieval (left), a conversational interface (center), and an agentic system selecting among tools and knowledge sources (right).

How This Book Is Organized

The book is structured in three parts that build on each other. Each chapter focuses on one retrieval capability and the techniques to implement it. The path is cumulative: later chapters assume you have worked through the earlier ones.

Part I: Foundations (Chapters 1-3)

The first three chapters establish the fundamentals that every retrieval system relies on.

Chapter 1: Classical Text Retrieval introduces the core problem: given a query and a collection of documents, find the relevant ones. We build from Boolean retrieval (exact keyword matching) through TF-IDF weighting to BM25, the statistical ranking model that still powers production search engines today.

Chapter 2: Performance Evaluation asks the question every retrieval engineer must answer: does this system actually work? We cover precision, recall, ranked evaluation metrics (MAP, NDCG), and the experimental methodology for benchmarking retrieval systems.

Chapter 3: Advanced Text Processing looks at what happens before retrieval: how raw text is transformed into features that improve search quality. Tokenization, stemming, compound handling, query understanding, and intent classification.

Part II: Search Systems (Chapters 4-7)

The middle chapters move from individual techniques to complete systems.

Chapter 4: Index for Text Retrieval covers the data structures that make retrieval fast: inverted files, posting lists, compression, and how systems scale from thousands to billions of documents.

Chapter 5: Semantic Search introduces the shift from keyword matching to meaning matching. We trace the path from latent semantic indexing through word embeddings to transformer-based dense retrieval.

Chapter 6: Vector Search addresses the infrastructure challenge: once documents are represented as vectors, how do we find nearest neighbors efficiently among millions of embeddings? Approximate nearest neighbor algorithms, quantization, and vector databases.

Chapter 7: Retrieval-Augmented Generation combines retrieval with large language models. We cover chunking strategies, query transformation, retrieval pipelines, and how to build systems that generate answers grounded in retrieved evidence.

Part III: Advanced Topics (Chapters 8-12)

The final chapters extend retrieval beyond text.

Chapter 8: Web Search adds link analysis (PageRank, HITS) and web-specific ranking signals to the retrieval stack.

Chapter 9: Content Analysis covers structural and metadata-based features for multimedia documents.

Chapter 10: Visual Features applies retrieval to images: color histograms, texture descriptors, shape features, and modern deep visual features.

Chapter 11: Audio Features extends retrieval to audio: perceptual features, musical features, and fingerprinting for music recognition.

Chapter 12: Video Structural Features addresses the temporal dimension: shot detection, motion features, and how to search within video.

What to Expect in Each Chapter

Every chapter starts with a concrete scenario that motivates the problem, then develops the solution through four layers:

1. **Concepts and examples.** We introduce each technique through a running example with real data, so you can see what the method does before we formalize how it works.
2. **Formal foundations.** Key formulas and models are stated precisely, with plain-English intuition alongside the math. You will know both what to compute and why.
3. **Practical implementation.** Code examples show how techniques translate into working systems. Where relevant, we reference production tools (Lucene, FAISS, LangChain) so you can connect theory to practice.
4. **Hands-on notebooks.** Interactive demos let you run the techniques yourself, experiment with parameters, and observe how changes affect retrieval quality.
5. **Quiz questions.** Each chapter includes multiple-choice questions in the Quiz App to test your understanding and prepare for the exam.

Each chapter ends with a **summary** that recaps the most important concepts, formulas, and insights in condensed form, useful for review and exam preparation. Summaries also point to further reading for those who want to go deeper.

What You Will Be Able To Do

By the end of this book, you will be able to:

- Build a text retrieval system from scratch using an inverted index
- Evaluate retrieval quality with precision, recall, and ranking metrics
- Process raw text into features that improve search quality
- Design a semantic search system using dense embeddings
- Construct a RAG pipeline that answers questions from a document collection
- Apply retrieval techniques to non-text media: images, audio, and video
- Choose the right retrieval architecture for a given problem

We begin with the simplest useful system (keyword search over a small collection) and end with systems that search across media types and generate natural-language answers. Let us start.

Further Reading

These four sources trace the same four-step progression this book follows, from keyword matching to generation. Read them for depth beyond the chapter treatment.

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. *Cambridge University Press*. Free online edition. The standard reference for classical text retrieval: indexing, Boolean and vector-space models, and evaluation. It underpins Chapters 1 to 4.
- Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. PDF. The definitive account of BM25, the statistical ranking model that anchors the second step of the progression and still serves as the standard baseline.
- Lin, J., Nogueira, R., & Yates, A. (2021). **Pretrained Transformers for Text Ranking: BERT and Beyond**. *arXiv preprint*. arXiv:2010.06467. A survey of how transformer models moved retrieval from surface matching to meaning, the third step covered in Chapter 5.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.11401. The paper that introduced RAG, the final step where retrieval feeds a language model to generate grounded answers, developed in Chapter 7.